



THALLYS HENRIQUE MARTINS

**APLICAÇÃO E AVALIAÇÃO DE GRANDES MODELOS DE
LINGUAGEM PARA CORREÇÃO E FEEDBACK DE
EXERCÍCIOS DE POO**

LAVRAS – MG

2026

THALLYS HENRIQUE MARTINS

**APLICAÇÃO E AVALIAÇÃO DE GRANDES MODELOS DE LINGUAGEM PARA
CORREÇÃO E FEEDBACK DE EXERCÍCIOS DE POO**

Trabalho de Conclusão de Curso apresentado
à Universidade Federal de Lavras, como
parte das exigências do Curso de Ciência da
Computação, para a obtenção do título de
Bacharel

Prof. Dr. Julio César Alves
Orientador

**LAVRAS – MG
2026**

**Ficha catalográfica elaborada pela Coordenadoria de Processos Técnicos da Biblioteca
Universitária da UFLA, com dados informados pelo(a) próprio(a) autor(a).**

Martins, Thallys Henrique

Aplicação e Avaliação de Grandes Modelos de Linguagem
para Correção e Feedback de Exercícios de POO / Thallys Hen-
rique Martins. 6ª ed. rev., atual. e ampl. – Lavras : UFLA,
2026.

21p. : il.

Tese(doutorado)–Universidade Federal de Lavras, 2026.

Orientador: Prof. Dr. Julio César Alves .

Bibliografia.

1. Inteligência Artificial. 2. Educação. 3. POO. 4. Feedback
Automatizado. I. Universidade Federal de Lavras. II. Título.

CDD 808.066

THALLYS HENRIQUE MARTINS

**APLICAÇÃO E AVALIAÇÃO DE GRANDES MODELOS DE LINGUAGEM PARA
CORREÇÃO E FEEDBACK DE EXERCÍCIOS DE POO**

**APPLICATION AND EVALUATION OF LARGE LANGUAGE MODELS FOR
CORRECTION AND FEEDBACK OF OOP EXERCISES**

Trabalho de Conclusão de Curso apresentado
à Universidade Federal de Lavras, como
parte das exigências do Curso de Ciência da
Computação, para a obtenção do título de
Bacharel

APROVADA em Data de Defesa.

Prof. MSc. Antônio Banca Um	UFM
Prof. DSc. João Banca Dois	FCO
Profa. Esp. Eliza Banca Três	BELMIS
Prof. Esp. Zorbison Uberplax IV	UFO

Prof. Dr. Julio César Alves
Orientador

**LAVRAS – MG
2026**

Espaço reservado à epígrafe.

(Autor Desconhecido)

RESUMO

A avaliação de exercícios práticos na disciplina de Programação Orientada a Objetos (POO) é fundamental para o aprendizado, mas exige um alto esforço do docente, especialmente em relação à consistência e clareza do feedback. Ferramentas de correção automática (como o JavaCorrige) são valiosas, mas muitas vezes limitadas na qualidade do retorno fornecido ao estudante. Com o avanço das Large Language Models (LLMs), torna-se viável a criação de um sistema de correção que ofereça feedback personalizado e sofisticado, superando as limitações dos sistemas tradicionais. A abordagem adotada neste projeto será a utilização da biblioteca LangGraph para criar um sistema de correção desacoplado, garantindo a flexibilidade técnica para comparar e alternar o modelo de IA (LLM) utilizado (ex: Gemini, OpenAI), o que é vital para a longevidade e validade da pesquisa.

Palavras-chave: programação orientada a objetos; grandes modelos de linguagem; langgraph; feedback pedagógico; avaliação de código.

ABSTRACT

The evaluation of practical exercises in Object-Oriented Programming (OOP) is fundamental for learning but demands high effort from instructors, particularly concerning consistency and feedback clarity. Automated correction tools (such as JavaCorrige) are valuable yet often limited in the quality of the feedback provided to the student. With the advancement of Large Language Models (LLMs), it becomes feasible to create a correction system that offers personalized and sophisticated feedback, overcoming the limitations of traditional systems. The approach adopted in this project involves using the LangGraph library to create a decoupled correction system, ensuring technical flexibility to compare and switch the AI model (LLM) used (e.g., Gemini, OpenAI), which is vital for the longevity and validity of the research.

Keywords: object-oriented programming; large language models; langgraph; pedagogical feedback; code evaluation.

SUMÁRIO

1	INTRODUÇÃO	7
1.1	Objetivos	8
1.2	Estrutura do Texto	8
2	REFERENCIAL TEÓRICO	9
2.1	Ensino de Programação Orientada a Objetos(POO)	9
2.2	Modelos de Linguagem de Grande Escala(LLMs)	9
2.3	Correção Automática de Código	9
2.4	IA como Tutor e Orquestração Pedagógica	10
2.5	Eficiência Computacional e Escalabilidade	10
2.6	A Biblioteca Langgraph	11
3	METODOLOGIA	12
3.0.1	Fase 1: Validação de Conectividade e Robustez do Cliente	12
3.0.2	Fase 2: Validação da Estrutura e Engenharia de Prompt	12
3.0.3	Fase 3: Validação da Orquestração com LangGraph	13
3.0.4	Fase 4: Gestão de Input e Leitura de Arquivos	13
3.0.5	Fase 5: Correção de Projetos Multi-Arquivo	13
3.0.6	Fase 6: Pré-processamento e Limpeza via Regex	14
3.0.7	Fase 7: Critérios de Viabilidade Econômica e Escalabilidade	14
4	RESULTADOS	16
4.1	Estudo de Caso I - Exercício Nave	16
4.1.1	Comparação Funcional por Estado de Correção	16
4.2	Estudo de Caso II - Exercício Casa	16
4.2.1	Comparação Funcional por Estado de Correção	17
4.3	Análise Qualitativa de Submissões Reais	17
4.3.1	Descobertas na Classe Nave	17
4.3.2	Descobertas na Classe Casa	18
4.4	Síntese da Análise Científica	19
5	APÊNDICE A - PROMPT DE CORREÇÃO UTILIZADO	20

1 INTRODUÇÃO

A crescente demanda por profissionais qualificados na área de tecnologia tem impulsionado a busca por métodos de ensino de programação mais escaláveis e eficientes. No contexto do ensino superior, as disciplinas de Programação Orientada a Objetos (POO) representam um marco na formação do estudante, exigindo a transição do pensamento puramente algorítmico para uma visão baseada em design, abstração e arquitetura de sistemas. No entanto, a avaliação qualitativa de exercícios nesse paradigma apresenta desafios significativos.

O processo de fornecer feedback detalhado e personalizado é uma tarefa que consome um volume considerável de tempo do docente. De acordo com a literatura (Pereira; Mello, 2025), a falta de um retorno pedagógico imediato e consistente pode prejudicar o ciclo de aprendizagem do discente, especialmente quando os erros cometidos não são apenas sintáticos, mas conceituais, envolvendo princípios fundamentais do paradigma, como encapsulamento e coesão (Cipriano; Alves, 2023).

Ferramentas de correção automática tradicionais, embora úteis para validar se um código compila ou passa em testes unitários, frequentemente falham em oferecer uma análise qualitativa profunda. Estas ferramentas costumam atuar como validadores estruturais, focando no "que" o código faz e negligenciando o "como" ele foi projetado. Com a recente popularização dos Grandes Modelos de Linguagem (LLMs), abre-se uma nova fronteira para a automação pedagógica, permitindo a geração de feedbacks que se assemelham à análise realizada por um tutor humano.

Entretanto, a integração direta de LLMs em sistemas de ensino esbarra em obstáculos técnicos, como a tendência desses modelos a gerar informações imprecisas (alucinações) e a necessidade crítica de supervisão humana para garantir a sustentabilidade pedagógica (Pereira; Mello, 2025). Além disso, a rápida evolução tecnológica torna arriscado o desenvolvimento de sistemas dependentes de um único provedor de inteligência artificial.

Este trabalho propõe a aplicação e avaliação de um sistema de feedback automatizado construído sobre uma arquitetura desacoplada. Utilizando o framework LangGraph para orquestração de estados e o modelo Gemini 2.5 Flash via SDK oficial. A pesquisa foca na capacidade de alternar entre diferentes modelos generativos, garantindo a longevidade da ferramenta e avaliando como a presença de uma solução de referência (gabarito) altera a profundidade qualitativa do feedback fornecido ao estudante de POO.

1.1 Objetivos

O objetivo deste trabalho é desenvolver e avaliar um protótipo de sistema de correção de código para exercícios de POO utilizando a biblioteca LangGraph, com foco na garantia de flexibilidade técnica e na qualidade do feedback gerado por IA. Para isso, os seguintes objetivos específicos foram estabelecidos:

- Desenvolver o fluxo principal de correção e análise utilizando a biblioteca LangGraph, garantindo a flexibilidade técnica para alternar e comparar diferentes Modelos de Linguagem Grande (LLMs).
- Automatizar a geração de anotações estruturais e gabaritos de correção do JavaCorrige por meio da IA, reduzindo o esforço do docente.
- Implementar um módulo de IA para enriquecer o relatório de resultados, transformando notas quantitativas em feedback qualitativo e claro para os estudantes.
- Utilizar a IA para realizar a correção propriamente dita do código, e integrar seus resultados com a correção estrutural do JavaCorrige para compor uma avaliação final mais completa.
- Validar a eficácia, precisão e consistência das correções e dos feedbacks gerados pela IA em um conjunto de exercícios reais de Programação Orientada a Objetos.
- Analisar e documentar os custos, as limitações e as licenças das LLMs disponíveis para garantir a viabilidade econômica do sistema a longo prazo.

1.2 Estrutura do Texto

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec a diam lectus. Sed sit amet ipsum mauris. Maecenas congue ligula ac quam viverra nec consectetur ante hendrerit. Donec et mollis dolor. Praesent et diam eget libero egestas mattis sit amet vitae augue. Nam tincidunt congue enim, ut porta lorem lacinia consectetur. Donec ut libero sed arcu vehicula ultricies a non tortor. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean ut gravida lorem. Ut turpis felis, pulvinar a semper sed, adipiscing id dolor.

2 REFERENCIAL TEÓRICO

Este capítulo apresenta o referencial teórico que fundamenta a pesquisa realizada. Nele, são discutidos os principais conceitos, teorias e trabalhos relacionados ao tema abordado, proporcionando uma base sólida para a compreensão do problema de pesquisa e a justificativa da metodologia adotada. A revisão da literatura é organizada de forma a destacar as contribuições mais relevantes e identificar lacunas que a presente pesquisa busca preencher.

2.1 Ensino de Programação Orientada a Objetos(POO)

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec a diam lectus. Sed sit amet ipsum mauris. Maecenas congue ligula ac quam viverra nec consectetur ante hendrerit. Donec et mollis dolor. Praesent et diam eget libero egestas mattis sit amet vitae augue. Nam tincidunt congue enim, ut porta lorem lacinia consectetur. Donec ut libero sed arcu vehicula ultricies a non tortor. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean ut gravida lorem. Ut turpis felis, pulvinar a semper sed, adipiscing id dolor.

2.2 Modelos de Linguagem de Grande Escala(LLMs)

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec a diam lectus. Sed sit amet ipsum mauris. Maecenas congue ligula ac quam viverra nec consectetur ante hendrerit. Donec et mollis dolor. Praesent et diam eget libero egestas mattis sit amet vitae augue. Nam tincidunt congue enim, ut porta lorem lacinia consectetur. Donec ut libero sed arcu vehicula ultricies a non tortor. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean ut gravida lorem. Ut turpis felis, pulvinar a semper sed, adipiscing id dolor.

2.3 Correção Automática de Código

Historicamente, o feedback automatizado em ambientes educacionais tem se concentrado em aspectos quantitativos, como a correção de respostas objetivas ou a validação de código por meio de testes unitários. Embora essas abordagens garantam agilidade, elas limitam-se a informar se um resultado está correto ou incorreto, falhando em fornecer a orientação peda-

gógica necessária para que o discente compreenda a origem do seu erro conceitual. Com o advento das LLMs, torna-se possível realizar uma análise semântica e qualitativa que atinja um nível de feedback que se aproxima do oferecido por um tutor humano.

A viabilidade técnica dessa evolução é validada por estudos como o de Wang (2023), que demonstrou ser possível automatizar o feedback docente com alta fidelidade ao utilizar modelos especializados. A arquitetura proposta alcançou uma pontuação METEOR (*Metric for Evaluation of Translation with Explicit Ordering*) superior a 96%, o que indica que as respostas geradas são quase indistinguíveis do julgamento de um professor especializado. O uso da métrica METEOR é fundamental para essa conclusão, pois ela avalia a qualidade do texto considerando sinônimos e a coesão lógica, garantindo que o sistema ofereça um suporte pedagógico robusto e confiável para o aprendizado de programação.

2.4 IA como Tutor e Orquestração Pedagógica

De acordo com Pereira e Mello (2025), o uso de métodos manuais de avaliação em cursos de larga escala frequentemente resulta em atrasos na correção, o que impacta negativamente a motivação e a participação dos discentes. Os autores defendem que as LLMs oferecem uma solução para este problema ao possibilitarem respostas em tempo real, garantindo que o retorno pedagógico seja imediato e relevante para o processo de aprendizagem.

No entanto, um dos maiores desafios pedagógicos reside em evitar que a IA forneça a resposta pronta, comprometendo o esforço cognitivo do aluno. Jacobs e Jaschke (2023) demonstram que, ao utilizar modelos avançados como o GPT-4, é possível mitigar quase integralmente essa limitação, fornecendo sugestões e orientações sem expor a solução final. Os autores ressaltam que, para manter essa eficácia, é vital evitar o envio de informações contraditórias à IA (como conflitos entre o código do aluno e o gabarito), o que reforça a necessidade de fluxos de dados controlados por orquestradores para garantir a integridade do papel de tutor.

2.5 Eficiência Computacional e Escalabilidade

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec a diam lectus. Sed sit amet ipsum mauris. Maecenas congue ligula ac quam viverra nec consectetur ante hendrerit. Donec et mollis dolor. Praesent et diam eget libero egestas mattis sit amet vitae augue. Nam

tincidunt congue enim, ut porta lorem lacinia consectetur. Donec ut libero sed arcu vehicula ultricies a non tortor. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean ut gravida lorem. Ut turpis felis, pulvinar a semper sed, adipiscing id dolor.

2.6 A Biblioteca Langgraph

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec a diam lectus. Sed sit amet ipsum mauris. Maecenas congue ligula ac quam viverra nec consectetur ante hendrerit. Donec et mollis dolor. Praesent et diam eget libero egestas mattis sit amet vitae augue. Nam tincidunt congue enim, ut porta lorem lacinia consectetur. Donec ut libero sed arcu vehicula ultricies a non tortor. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean ut gravida lorem. Ut turpis felis, pulvinar a semper sed, adipiscing id dolor.

3 METODOLOGIA

Neste capítulo, descrevem-se os procedimentos metodológicos e as etapas técnicas que compuseram a prototipação inicial do sistema. A metodologia foi dividida em sete fases incrementais de validação, visando garantir a robustez, a precisão e a viabilidade da solução proposta.

3.0.1 Fase 1: Validação de Conectividade e Robustez do Cliente

O passo inicial teve como objetivo estabelecer a comunicação funcional entre o ambiente Python e o modelo `gemini-2.5-flash`.

1. **Conectividade:** Utilizou-se a biblioteca oficial do Google GenAI para estabelecer o canal de comunicação via API.
2. **Segurança:** Validou-se a utilização de arquivos de ambiente (`.env`) para o armazenamento seguro da chave de API (`GEMINI_API_KEY`).
3. **Resiliência:** Implementou-se a lógica de *Exponential Backoff*. Esta função demonstrou ser crucial para permitir que a aplicação se recupere automaticamente de limites de taxa (*rate limits*) temporários, garantindo a continuidade do serviço durante picos de requisição.

3.0.2 Fase 2: Validação da Estrutura e Engenharia de Prompt

Nesta fase, buscou-se garantir que a saída da LLM obedecesse a uma estrutura rígida, essencial para a tomada de decisão automatizada no fluxo orquestrado. O prompt foi projetado para impor uma persona de docente e uma estrutura de saída que atua como o *Gatekeeper* do sistema, conforme detalhado na Tabela 1.

Os testes iniciais desta fase confirmaram que o modelo é capaz de ir além da validação do enunciado, sugerindo boas práticas de POO, como a criação de construtores e a validação de entradas negativas.

Tabela 3.1 – Estrutura de Saída Rígida e Relação com a Orquestração

Estrutura de Saída Exigida	Função na Orquestração (LangGraph)
1. Avaliação	Utilizada como critério de controle para decidir o próximo nó do grafo.
2. Justificativa	Fornece o feedback qualitativo pedagógico ao discente.
3. Sugestão de Correção	Provê pistas técnicas para auxiliar o aluno sem revelar a solução.

3.0.3 Fase 3: Validação da Orquestração com LangGraph

Esta etapa testou a execução sequencial do nó de correção dentro de um grafo de estados. O objetivo foi garantir o processamento robusto de diferentes entradas de código de forma diferenciada. A execução sequencial de casos de teste (Correto, Errado e Parcialmente Correto) validou a detecção de falhas graves de POO, especialmente em conceitos de **Encapsulamento** e lógica de classe.

3.0.4 Fase 4: Gestão de Input e Leitura de Arquivos

Avaliou-se a melhor estratégia para a entrada de dados do mundo real.

- **Implementação:** A leitura de enunciados (.txt) e códigos do aluno (.java) foi realizada via I/O nativo do Python, integrando o conteúdo diretamente no estado inicial do LangGraph.
- **Estratégia de Envio:** Concluiu-se que o envio do código como texto formatado (*code fences* Markdown) é a prática mais eficiente, evitando a sobrecarga de codificação Base64 e garantindo que o modelo interprete corretamente a linguagem Java.

3.0.5 Fase 5: Correção de Projetos Multi-Arquivo

Testou-se a capacidade da LLM de correlacionar e analisar a consistência entre múltiplas classes. A estratégia de concatenação de arquivos com delimitadores claros demonstrou eficácia na identificação de erros de **Consistência Arquitetônica**, como a incompatibilidade

entre chamadas estáticas e métodos de instância em classes auxiliares, superando a barreira do contexto de código distribuído.

3.0.6 Fase 6: Pré-processamento e Limpeza via Regex

Uma etapa fundamental para a eficiência do sistema foi a implementação de uma rotina de pré-processamento de texto. Utilizou-se a biblioteca `re` do Python para aplicar Expressões Regulares (Regex) capazes de limpar o código-fonte antes do envio à API.

A lógica de limpeza opera em três níveis:

1. **Remoção de Comentários de Bloco:** Utiliza-se o padrão `r'\/\*..*?\*/'` com a *flag* `DOTALL` para identificar e excluir blocos delimitados por `/* */` independentemente do número de linhas.
2. **Remoção de Comentários de Linha:** Aplica-se o padrão `r'\/\s.*'` para eliminar anotações de linha simples.
3. **Compactação de Linhas Vazias:** Após a remoção dos caracteres, o script realiza uma operação de *join* em lista, removendo todas as linhas que resultaram em espaços em branco.

Esta técnica foi implementada para reduzir o consumo de tokens ao enviar o código para correção pela LLM.

3.0.7 Fase 7: Critérios de Viabilidade Econômica e Escalabilidade

A análise de viabilidade focou no modelo de preços *pay-as-you-go* da API Gemini.

1. **Modelo Flash:** O `gemini-2.5-flash` foi identificado como o melhor custo-benefício para tarefas de alto volume.
2. **Limites de Taxa:** Identificou-se que o *Free Tier* (20 requisições diárias) é impraticável para uso acadêmico. A solução metodológica para a escalabilidade é a ativação do faturamento no Google Cloud, o que eleva as cotas de RPM (Solicitações por Minuto) e TPM (Tokens por Minuto).

3. **Privacidade:** O plano pago garante que os dados e prompts não sejam utilizados para treinamento dos modelos do Google, atendendo a critérios de segurança de dados.

Tabela 3.2 – Comparação de Modelos para Correção de Baixo Custo (Base: 1M Tokens)

Modelo	Entrada (US\$)	Saída (US\$)	Vantagem
DeepSeek-V3.2	\$0.28	\$0.42	Menor custo de saída.
GPT-4o mini	\$0.15	\$0.60	Alta velocidade e baixo custo de entrada.
Gemini 2.5 Flash	\$0.30	\$1.25	Ampla janela de contexto e orquestração nativa.

4 RESULTADOS

Este capítulo apresenta a análise experimental da prototipação inicial, avaliando como a presença ou ausência de uma solução de referência (gabarito) altera a natureza da correção e a eficácia do feedback gerado pela IA.

4.1 Estudo de Caso I - Exercício Nave

A avaliação dos cenários de teste para o exercício "Nave" revelou padrões distintos na forma como a IA processa as informações baseando-se na ancoragem do gabarito.

4.1.1 Comparação Funcional por Estado de Correção

- **Cenário Correto:** Com o gabarito, a IA valida a equivalência técnica e confirma a concordância com valores exatos previstos pelo docente. Sem o gabarito, o modelo realiza uma validação funcional autônoma baseada apenas no enunciado.
- **Cenário Errado:** A presença do gabarito permitiu à IA identificar que o arquivo enviado era idêntico à versão inicial (não modificado), inferindo o estado base através da comparação direta. Na ausência do gabarito, a avaliação concentrou-se estritamente na inexistência técnica dos componentes solicitados.
- **Cenário Parcialmente Correto:** Com o suporte do gabarito, a IA identificou com precisão os itens específicos do enunciado que não foram satisfeitos. Sem o suporte, o modelo realizou uma análise lógica interna para apontar inconsistências.

4.2 Estudo de Caso II - Exercício Casa

Os testes com o exercício "Casa" aprofundaram a compreensão sobre a sensibilidade da IA a variações paramétricas e controle de estado em Programação Orientada a Objetos.

4.2.1 Comparação Funcional por Estado de Correção

- **Cenário Correto:** Com o gabarito, a IA classificou divergências numéricas triviais (ex: distância de movimento de -310 vs -300) como aceitáveis, mantendo o foco na funcionalidade e na orquestração de eventos baseada em estados (booleano `porDoSolRealizado`).
- **Cenário Errado:** A presença do gabarito permitiu confrontar o código com a arquitetura de referência, detectando falhas de sintaxe e omissões estruturais. Sem o gabarito, o modelo atuou prioritariamente como um analisador de compilação, focando em erros que impediriam a execução.
- **Cenário Parcialmente Correto:** A utilização do gabarito permitiu à IA identificar com precisão a omissão de métodos de modularização específicos (como o `porDoSol`) exigidos na arquitetura. Sem a referência, a IA detectou a falha lógica de sequenciamento, mas com menor detalhamento sobre a estrutura esperada.

4.3 Análise Qualitativa de Submissões Reais

Após a validação controlada, o sistema foi testado com submissões reais de alunos. Esta etapa permitiu identificar nuances adicionais na correção e limitações técnicas do modelo Gemini 2.5 Flash.

4.3.1 Descobertas na Classe Nave

- **Vulnerabilidade à Indentação (Alucinação Sintática):** Identificou-se que a IA é sensível à forma visual do código. No caso de chaves `{}` "empilhadas" ou desalinhadas, o modelo tende a alucinar erros de sintaxe (como classe não fechada).
- **Oscilação de Rigor (Flexibilidade):** Observou-se que a IA pode variar o nível de exigência em relação a padrões de texto. Enquanto em alguns casos penaliza a falta de caracteres específicos (como o hífen), em outros aceita formatos alternativos (parênteses) como "corretos", desde que a semântica seja mantida.

- **Fidelidade ao Enunciado:** Identificou-se que a IA prioriza o "sucesso funcional" do código sobre a "exatidão estética" em submissões que demonstram alta organização.
- **Rigor de Interface (Clean Code):** A IA demonstrou uma tendência a remover métodos que não foram explicitamente solicitados (como um getter extra em uma submissão de um aluno). Confirmando que a IA não avalia apenas a sintaxe, mas aplica conceitos de POO, como a minimização da interface, garantindo que o objeto exponha apenas o necessário para cumprir sua responsabilidade.
- **Desafio dos Exercícios Evolutivos:** Identificou-se que a IA pode sinalizar "erros de progressão" quando o aluno apresenta a solução final que naturalmente substitui uma etapa anterior.

4.3.2 Descobertas na Classe Casa

- **Lógica no Construtor:** A IA demonstrou alto valor pedagógico ao identificar como má prática a chamada de métodos dentro do construtor. O modelo reforça que o construtor deve ser restrito à inicialização do estado.
- **O Problema do Rigor Geométrico:** Identificou-se que a IA tende a penalizar divergências de coordenadas, cores e tamanhos (ex: 190 vs 180, Azul vs Amarelo, 80 vs 60), como o enunciado não restringiu esses valores, o feedback da IA torna-se desencorajador. Isso prova que o sistema de correção precisa de uma instrução (Prompt) que ensine a diferenciar erros de arquitetura graves de escolhas estéticas irrelevantes.
- **Validação de Assinatura e Nomenclatura:** A IA mantém o rigor sobre os nomes de atributos solicitados (ex: morador vs pessoa), o que é fundamental para a padronização de grandes projetos.
- **Violação da Responsabilidade Única (Coesão):** Ponto recorrente onde a IA identifica a "mistura de responsabilidades" (ex: mover o morador dentro do método do sol).

4.4 Síntese da Análise Científica

O experimento consolidado com os exercícios Nave e Casa comprovou três pilares fundamentais para a eficácia do sistema:

- **Precisão Técnica:** A IA demonstrou robustez pedagógica ao validar conceitos avançados de POO, incluindo a gestão de estados de objetos e a interação entre múltiplas classes.
- **IA como “Detetive”:** O gabarito atua como um ponto de referência essencial, permitindo que a IA diferencie entre erros conceituais graves e variações criativas menores do estudante.
- **Valor Pedagógico Superior:** Conclui-se que, sem gabarito, a IA atua apenas como analisadora de compilação; com o gabarito, ela ascende ao papel de analisadora de arquitetura, confrontando o design do aluno com as expectativas do professor.

5 APÊNDICE A - PROMPT DE CORREÇÃO UTILIZADO

Abaixo apresenta-se a estrutura exata das System Instructions utilizadas para orquestrar o comportamento da IA durante o processo de correção:

Listing 5.1 – Instrução de Sistema para Correção Automatizada

```
Você é um Professor de Programação Orientada a Objetos (POO) da
UFLA. Avalie o código Java do aluno considerando o
enunciado. Forneça feedback construtivo e educativo, focado
em princípios de POO (Encapsulamento, Herança, Lógica).
Sua resposta DEVE ser um JSON válido, SEM TEXTO ANTES OU DEPOIS
, e seguir EXATAMENTE este schema:
{
  "avaliacao": "Certo" | "Errado" | "Parcialmente Certo",
  "justificativa": string,
  "dica_correcao": string,
  "sugestao_correcao": string
}
No campo 'sugestao_correcao', coloque o código Java corrigido e
inclua comentários explicativos sempre que possível para
ajudar o aluno a entender as correções e boas práticas. Não
inclua mensagens de parabéns ou explicações fora do código.
Se a sugestão envolver mais de um arquivo, separe cada
arquivo usando exatamente o padrão:
// --- ARQUIVO INÍCIO: NomeDoArquivo.java ---
(código do arquivo, com comentários explicativos se necessário)
// --- ARQUIVO FIM: NomeDoArquivo.java ---
No campo 'dica_correcao', coloque as orientações e dicas para o
aluno corrigir o código, sem incluir o código em si. Use
```java apenas no prompt, não na resposta JSON. Responda
integralmente em português. Não adicione explicações fora do
JSON.
```

Quando uma solução de referência (gabarito) é fornecida ao sistema, a instrução abaixo é concatenada ao prompt para garantir a ancoragem técnica:

### Listing 5.2 – Instrução Adicional para Ancoragem por Gabarito

```
Avalie o código do aluno COMPARANDO DIRETAMENTE com o gabarito
acima. Considere o gabarito como a única solução correta.
Aponte diferenças, similaridades e se o código do aluno está
igual, melhor ou pior que o gabarito. Siga a estrutura rí
gida definida no System Instruction.
```

